GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unidad 10 — ACL y NAT
Bloque 1: quién pasa y quién no

1r CFGS ASIR · Planificación y Administración de Redes

Sergi García Barea

Las ACL: la idea en una frase

> Una **ACL** (Access Control List) es el **portero del router**: una lista ordenada de reglas que se lee **de arriba abajo** y que decide, paquete a paquete, si **pasa o se descarta**. Sin ACL, todo el tráfico entra — con ACL, tú escribes la puerta.

Hasta UD09 movías tráfico con rutas — el router buscaba «a dónde va». Ahora controlas «**quién tiene permitido ir**». Es el salto de administrador a **administrador con criterio de seguridad**: la misma tabla de ruteo, pero filtrando por origen, destino, puerto o protocolo. En este bloque: cómo evalúa el router una lista, cómo se escriben las máscaras wildcard y qué tipos de ACL hay.

Para qué sirve: tres usos protagonistas

1. **Filtrar tráfico** — el uso estrella: «la VLAN de invitados no llega a la LAN corporativa».
2. **Clasificar**: seleccionar qué tráfico merece otros servicios (NAT, QoS, route-maps). La ACL no filtra: *lista* qué paquetes cumplen.
3. **Proteger el acceso**: con `access-class` en las líneas VTY decides **desde qué IPs se puede hacer Telnet/SSH al router**.

ℹ️ Aviso de nomenclatura: **permit** = pasa, **deny** = se descarta. Los usos 2 y 3 no «bloquean» nada — usan la lista como **criba**: lo que cumple, gana el servicio.

La regla de oro: se lee en orden e implicit deny

1. El router **lee las reglas de arriba abajo** y aplica la **primera que case**: no sigue leyendo. Cambiar el orden cambia el resultado.
2. Si **ninguna regla casa**, entra el **deny implícito**: la última regla invisible de toda ACL es `deny any`. *Una ACL vacía bloquearía internet entera*.
3. Lectura mental: «¿Primera que aplique? Permito o deniego y **termino**. No hay segundas oportunidades».

⚠️ **Consecuencia operativa:** las reglas **más específicas van arriba** y las genéricas abajo. Si pones `permit any` primero, ninguna regla que va debajo se leerá nunca.

Wildcards: la máscara que conoces, al revés

Máscara normal	Wildcard	Qué casa
`255.255.255.255`	`0.0.0.0`	Un host exacto (`host 192.168.10.5`)
`255.255.255.0`	`0.0.0.255`	Toda la red /24
`255.255.0.0`	`0.0.255.255`	Todo un /16
—	`255.255.255.255`	`any` : «cualquiera»

ℹ️ Truco: **wildcard = 255.255.255.255 – máscara**. En OSPF en UD09 ya la usaste en el `network … 0.0.0.3 …` — aquí es exactamente la misma pieza.

Los tipos: estándar y extendida

Estándar (1–99)

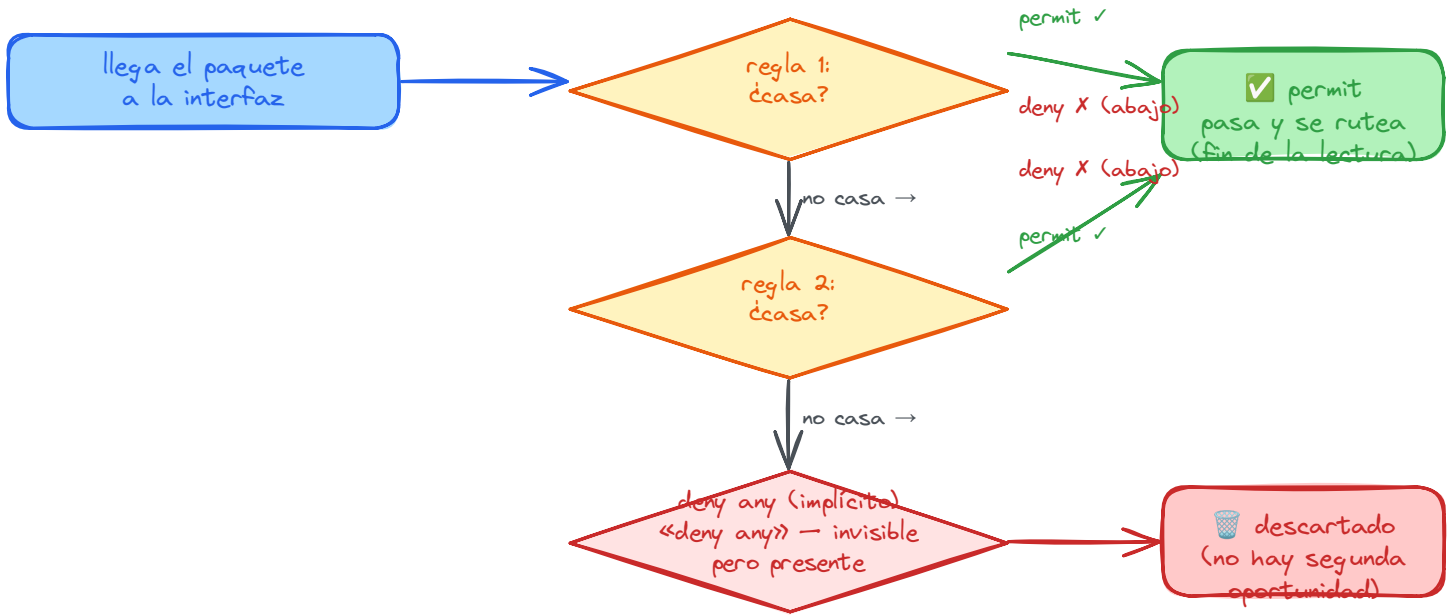
Filtra **solo por origen**: «los PCs de la VLAN 20 no pasan». Es la pregunta de «**quién**». Compacta pero torpe: no mira destino ni puerto.

Extendida (100–199)

Filtra por **origen + destino + protocolo + puerto**: «la VLAN 20 puede navegar (TCP 80/443) a la web interna, pero no usar RDP». Precisión de cirujano.

ℹ️ También existen las **named** (con nombre en vez de número, se editan línea a línea) y el **número** solo es una convención de clasificación — el comportamiento lo define el contenido.

La tabla de decisión del portero



Se lee de arriba abajo: la regla 1, la 2... la primera que casa decide y se acaba.
ıguna casa, el deny implícito descarta: toda ACL acaba con deny invisible — por eso los permit valientes van bien arrit

Dónde colocarla: la regla de proximidad

Tipo	Dónde aplicarla	Por qué
Estándar	**Cerca del destino**	Solo ve el origen: si la pones lejos, cortas de más
Extendida	**Cerca del origen**	Describe todo el viaje: filtra *antes* de recorrer la red en balde

⚠️ Colocar es **medio del trabajo**: una ACL bien escrita en la interfaz equivocada protege... a nadie. Y **dónde y en qué sentido** filtra: `in` mira lo que entra a la interfaz, `out` lo que sale de ella.

La sintaxis estándar de un vistazo

```
Router(config)# ACL estándar 10 en un /24

Router(config)# access-list 10 permit 192.168.10.0 0.0.0.255
Router(config)# access-list 10 deny   192.168.20.0 0.0.0.255
Router(config)# access-list 10 permit any
Router(config-if)# ip access-group 10 in
```

1. `access-list 10`: es una estándar (1–99). Cada `access-list` **añade una línea** al final — no se edita en sitio.
2. `permit / deny` + origen **con wildcard**. `any` equivale a `0.0.0.0 255.255.255.255`.
3. La última línea de la lista (implicit deny) **no se ve**, pero está: si te olvidas un `permit`, bloqueas.

Mini-chequeo: el portero

🧠 Responde rápido:

1. ¿Qué pasa si **ninguna** regla casa con un paquete?
2. Una ACL con `permit any` en la línea 1 y `deny host x` en la 2: ¿se aplica el deny?
3. ¿La wildcard de la máscara `255.255.252.0`?

1. Se cae al **deny implícito**: el paquete se descarta. · **2.** No: la primera que casa **termina la evaluación** — el host queda permitido por la línea 1. El orden es todo. · **3.** `0.0.3.255` — máscara invertida (255.255.255.255 − máscara).

Logros y siguientes pasos

Logro	Cómo conseguirlo
🏅 **Lector de evaluación**	Decidir paquete a paquete si pasa o no, con el orden en la cabeza
🏅 **Domador de wildcards**	Escribir la wildcard correcta para cualquier máscara, sin dudar
🏅 **Elección de tipo**	Ante un requisito, decir si basta la estándar o hace falta extendida
🏅 **Ubicación correcta**	Decidir interfaz y dirección (in/out) de una ACL nueva

ℹ️ **Bloque 2:** ACLs named y extendidas a la obra, VTY protegida con `access-class` y verificación. Para practicar: boletines U10 en Packet Tracer.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //